



Automating Safety Critical Software Engineering

Agentic Optimisation and Transformation

Name: Oliver Haythornthwaite

Student ID: s479832

Department: Applied Artificial Intelligence

Date: 15/08/2026

Information categorisation:

Open

Contents

Section	Page Range	Word Count
Background and Research Aims	3-5	1194
State of the Art	5-9	1686
Scope of Applicability and Methodology Control	10-11	404
Methodology Review 1	12-14	877
Methodology Review 2	15-17	858
Methodology Application and Future Development	18-20	747
References	21-23	N/A
Total Word Count: 5766		

Background and Research Aims

Introduction

Safety critical software defects or failures constitute a risk to the loss of a platform or human life. Assurance of such a system must demonstrate the development process has systematically prevented, detected and resolved defects and failure modes.

In civil airborne safety critical software development, project level adherence to RTCA DO-178C [1] is demonstrated through the application of standards derived processes. Planning documentation defines how the objectives concerning lifecycle artefacts and those in supplementary standards including DO-330, and DO-331 [2, 3] are satisfied. The required evidence scales with the Design Assurance Level (DAL), where greater effort is required where software failure could contribute to a catastrophic condition. Defence Standard 00-055 Part 1 [4] recognises RTCA DO-178C [1] as an acceptable means of demonstrating compliance for the mitigation of such conditions. Further the FAA Advisory Circular 20-115D [5] substantiates RTCA DO-178C [1] suitability as a means of compliance, reiterating the responsibility of the integrator, in the satisfaction of applicable objectives.

The execution of these processes to produce such artefacts is highly parallelisable, and automatable dependent on the presence of well-defined requirements at the system and dependency domain level. However, the scope of this issue extends beyond engineering process automation, dependent on the DAL, a certifiable project can require the production of all lifecycle artefacts defined within Table 1.

DO-178C Reference	Lifecycle Component	Lifecycle Process	DAL
11.1	Plan for Software Aspects of Certification (PSAC)	Planning	A-D
11.2	Software Development Plan (SDP)	Planning	A-D
11.3	Software Verification Plan (SVP)	Planning / verification	A-D
11.4	Software Configuration Management Plan (SCMP)	Planning / Configuration	A-D
11.5	Software Quality Assurance Plan (SQAP)	Planning / QA	A-D
11.6	Software Requirements Standards	Development standards	A-C
11.7	Software Design Standards	Development standards	A-C
11.8	Software Implementation Standards	Development standards	A-C
11.9	Software Requirements Data	Requirements Development	A-D
11.10	Design Description	Software design	A-C
11.11	Source Code	Coding	A-C
11.12	Executable Object Code	Integration	A-D
11.13	Software Verification Cases and Procedures	Verification	A-D
11.14	Software Verification Results	Verification	A-D
11.15	Software Life Cycle Environment Configuration Index	Configuration	A-D
11.16	Software Configuration Index	Configuration	A-D
11.17	Problem Reports	Configuration	A-D
11.18	Software Configuration Management Record	Configuration	A-D
11.19	Software Quality Assurance Records	QA	A-D
11.20	Software Accomplishment Summary (SAS)	Certification liaison	A-D
11.21	Bi-Directional Traceability Evidence	Development / verification	A-D
11.22	Parameter Data Item File	Development / verification	(Where used)

Table 1 - RTCA DO-178C Lifecycle Components

Environmental and Project Context

Bi-directional traceability requires a cohesive requirement decomposition, supported by through life trace management support, offered in toolsets such as SCADE. This lifecycle, when employed using an MBSE approach, is already optimised from an assurance perspective, as bounded credit for DO-331 can be claimed, via the vendor supplied KCG qualified code generation capability. This credit does not remove the burden of proof around requirement quality, model implementation correctness, or the satisfaction of objective based processes [3, 6, 7].

Manual process automation is facilitated by the characteristics of the supportive standards; the work is requirements driven, evidence rich and supported by deterministic toolsets, maintaining a standardised approach to the delivery of artefacts. Large Language Models (LLMs) can ingest natural language, code and prompted context, while Retrieval Augmented Generation (RAG) can instead route responses through the project's corpus of information, rather than relying upon model parameters alone [8]. Agentic orchestration enables model instantiations to plan, use toolsets and coordinate other highly specialised/constrained agents for the fulfilment of a task [25].

Considering the proprietary nature of MBSE tools, and to integrate such tools into an agentic workflow, they shall be used to produce associated artefacts, but use must remain compliant with license restrictions and fair use stipulations. A process which considers impacts to established safety case driven evidence substantiation, must be prioritised, to not increase certification effort. The associated airworthiness authority must validate that the defined Acceptable Means of Compliance is substantiated by impact appropriate processes.

Risk and Security

Agentic approaches to software development are rapidly evolving. With the democratisation of Artificial Intelligence (AI) bringing more memory/cache efficient frontier models into the scope of local or on-prem execution, the value proposition of developing a proprietary and certifiable Agentic Model Based Software Engineering (AMBSE) pipeline increases.

However, the desirable benefits offered by the technology, introduce technological and commercial risk. The underlying principles of transformer architectures, means LLM output is probabilistic, sensitive to prompt correctness, and that they can fluently and plausibly answer a question incorrectly. Generated artefacts may therefore appear correct, but hide hallucination induced traceability, functionality or logical errors within a plausibly structured and presented artefact.

Further, post deployment learning and even conversation context, can upset the determinism of answers and reduce repeatability. Should proper consideration not be given to hosting or security concerns, Intellectual Property (IP) or classified information could be exposed through logging, callbacks, adversarial prompting or poisoned retrieval content or mechanisms [21, 22, 27].

On premises deployment permits tighter control of data residency and disclosure, however they introduce upfront hardware, maintenance and security costs [29]. Quantisation can reduce RAM and VRAM requirements, but smaller or lower precision models can incur performance drop offs and inaccuracy. Access to frontier models provides additional capability where tool integration, reasoning and parameter size is concerned, however such usage potentially risks unauthorised disclosure of export controlled, or otherwise classified information. Schedule savings can only be realised, once model hosting, integration, baselining and evaluation have been performed.

Information categorisation:

Open

Regulatory and Organisational Considerations

The regulations applicable to the utilisation of AI as a development aid, and around the airborne deployment of AI distinctly identify associated challenges. It is not the aim of this report to suggest a means for the assurance or expedition of deployed AI, instead this report is primarily concerned with AI enabled development. Guidance within UK MOD JSPs and the EASA Artificial Intelligence Concept Paper (Proposed Issue 3) [9, 24], whilst concerned with deployed AI, relies upon concepts such as operational domains/design domains, human factors, governance and safety cases, which are also applicable to the use of LLMs as development aids. EASA and MOD guidance provides useful governance and as such, their recommendations shall be adhered to where applicable, but DO-178C, DO-330 and DO-331 remain as the basis for assurance through Defence Standard 00-055 Part 1 [1-5].

Where no certification credit is claimed for the development aid and where output is fully reviewed via an independently controlled review process, tool qualification may not be required [1, 2, 5]. The cost and engineering ability required to perform such reviewing, is already considered at the project planning stage, according to any assumed DAL rating [1].

A practicable implementation route requires integration with established and qualified toolsets. LLM produced model code, can be imported into SCADÉ, from which qualified code is generated, using SCADÉ's KCG facility. However, the correctness of any generated model must still be verified against requirements [3, 6].

The structure of engineering teams is formed according to the needs of a specific project, its associated budget and timeframe. It is the intent of this review to maintain such a structure, to retain attribution, for the establishment of assurance. It is recognised human review is most effective when involving Suitably Qualified and Experienced Personnel (SQEP), however such reviewers require sufficient context, time and capability to detect a plausibly presented, but concealed defect.

Automation bias can cause a tendency to omit information or contrary evidence, primarily whilst under pressure, workload or when using poorly defined performance metrics [10].

Therefore, any safe process transformation, must provably preserve the quality, independence and traceability afforded by any previous iteration. The feasibility of agentic engineering, will be evaluated through analysis of potential improvements which can be gained through the allocation of any number of agentic engineers, prompted, guided and validated in a custom framework by SQEP.

Accordingly, this review establishes if a human led, LLM enabled lifecycle, or a gated Agentic lifecycle can reduce elapsed time and engineering effort within an MBSE, DO-178C/DO-331 compliant development schedule. The achievement of this effort must be balanced by an evaluation of compliance, the completeness of objective related evidence, the absence of defects, traceability and independence. The intended contribution is the design of process acceleration and the establishment of an empirical basis to allow the safe transformation of established engineering processes.

State of the Art

Assurance Led Manual Safety Critical Software Development

The operational context, monetary value and consequences of systems failure, inform the methods used to assess the suitability and correctness, of software design, implementation and verification. Historically, software written in assembly received additional review around memory usage, efficient instruction, supported by flight testing to substantiate the means of compliance. With the observed increase in software complexity, permutatively exhaustive verification, and formal language implementation approaches become infeasible. DO-178A defined processes, requiring the correct documentation, review and testing of software, across a structured lifecycle. Following iterations, DO-178B and DO-178C, shifted to promote compliance to objectives (with objective compliance scaled according to Design Assurance Levels (DAL), through the collection and demonstration of evidence, defined in targeted plans and standards documentation [1, 5].

The traditional software lifecycle, when executed using Agile principles, is driven by the delivery of artefacts, which provide the necessary evidence, to claim compliance, against pertinent plans and standards documentation, implementing the means of compliance for high level standards such as DO-178C.

At each stage of delivery, artefacts are reviewed for compliance against plans and standards, and where relevant, to confirm their functional correctness as per the stated requirements. This is a sequential process, where system level requirements (and associated de-risking activities) are produced, as pre-requisite deliverables, allowing the sequential commencement of High Level Requirement (HLR) and Low Level Requirement (LLR) implementation.

Processes which increase the quality of any delivered artefact, tend to cost the project more time and money, according to an increase in the Design Assurance Level (DAL) associated with the project / domain, according to additional objectives, verification quality and independence requirements [1].

Consequently, high integrity software would see considerable cost savings from complete automation, whilst incurring the highest potential penalty as a consequence of failure, in the loss of human life. For this reason, the integration of AI/ML into the domain of safety critical software engineering, remains subject to concern and scepticism. No generally accepted means of compliance, for deployed and fully autonomous AI/ML has been agreed, which current guidance maintaining a position of caution, advising against fully autonomous or human out of the loop operation (Level 3 AI - EASA) [9, 23].

LLM Supported Safety Critical Software Development

Current software engineering literature, anecdotal evidence and the prevailing public opinion align in the identification of LLMs as a transformative offer of capability. RAG integration indeed offers a practical route for the grounding of models, in the corpus of project data, bounding responses in that which can be observed as fact [8]. Safety critical LLM development aids should accordingly include increased logging, source identification and integration with configuration control. Traceability and explicit location references for any decision making shall be treated as an auditable configuration artefact.

Increases in productivity have been measured across the literature, but the magnitude of results is dependent on the setting and context. As part of a bounded programming task, developers were asked to utilise GitHub Copilot, with a treatment group completing the task 55.8% faster than a control group [11]. Despite this, a randomised study querying open-source software developers, working with mature, known codebases in 2025 found completion time increased by 19%, contradicting expected increases in productivity, which were higher [12].

These contrasting results indicate that AI excels at greenfield programming tasks, whereas additional reasoning and context is required to parse and measurably improve a highly optimised code base, which is an expected outcome. Indeed, any safety critical improvement would be mired by additional QA confirmation delays, review timeframes, timing analysis and failure mode analysis. These studies indicate why this experiment must measure human active development time, waiting time, review time and rework, as separate considerations, rather than apply a single productivity increase factor, to any later scheduling.

Quality focused studies on LLM enabled development, purport additional insecure coding risks. Perry et al. [13] found participants in a study produced less secure solutions for a given task, whilst exhibiting more confidence in the security and correctness of their implementation. Reasoning ability in models, is not enough to infer the additional context or instruction, where poor prompting or key details are omitted, in a safety critical context. Model knowledge of planning and standards documentation and all pertinent requirements is not enough to confirm usage, orchestration or 'master-prompting' must be used to ensure the context window contains the complete extent of information pertinent to any task. NASA's review of LLM usage in assurance case argument composition indicated available studies do not demonstrate general efficacy in the production of complete safety arguments, in comparison with human produced examples, with time and cost savings being compromised by rework cycles [14].

Agentic Safety Critical Software Development

Agentic approaches extend LLM usage through the enablement of planning, memory, tool usage and role definition. The original SWE-bench study [15] found the best agentic model only resolved 1.96% of 2294 issues, whereas a localisation-repair-validation approach later achieved a 32% completion rate, against the comparable 'SWE-bench Lite' benchmark [16].

These benchmarks provide a decent comparative measure, but do not prove compliance to outlined programming standards or prove additional defects are identified and resolved, which can increase risk of failure. The proposed Agentic FEL must be designed such that related system components can be reviewed and evaluated together, to minimise human delays, whilst identifying and removing defects, achievable through SW/SW integration and Unit testing.

It is evident that the tools necessary to permit agentic interaction within a local machine, exist and are freely available. Orchestration frameworks such as MetaGPT [17] and ChatDev [18] organise model instances into roles, for the coordination and parallelisation of tasks, resembling a deployed software engineering team. It is this similarity, which has in part prompted an investigation into the optimal structure of an agentic software team. The technical feasibility of agentic employment for the development of software, against stated requirements, has been demonstrated to a high level, however the holistic performance of these systems in the development of software to a DAL B standard, has not been sufficiently performed.

Orchestration frameworks each operate with unique characteristics and capabilities. Multi-role, singular agent deployment has yielded promising results, evidenced by 'Agentless', where localised, repair and validation loops achieve strong results, without the specialisation or deployment of multiple agents. Context window and token budget restrictions must be assessed against the length and complexity of the projects plans, standards and requirements.

Prototyping has confirmed the feasibility of LLM integration with SCADE Suite. Ansys provides the SCADE Python APIs [19], by which process automation can be achieved. Due to the use of additional toolsets, the security boundary of agentic interaction extends, it is therefore prudent to isolate such a model to within the confinement of a virtual machine or sandbox [21, 22, 27]. This isolation aligns with the proposed process, where the fulfilment of a prior action would trigger agentic execution of the next sequentially blocked item, in a sandbox environment within a virtual machine. NIST defines data governance restrictions which shall be followed in any further deployment of this framework [21]. The NIST SP 800-218A document applies secure development practices to generative AI and model-dependent systems [22]. Relevant controls are included in the Agentic FEL methodology discussion.

Regulatory & Human Factors Position

EASA's Proposed AI Concept Paper [9] defines an approach to software assurance based upon the satisfactory consideration of concepts such as operational and design domains, learning assurance, explainability, risk mitigation and human factors. The UK Military Aviation Authority (MAA) administrative Regulatory Notice 2025/04 advises similar considerations, including defined operational domains, learning related assessments and architectural mitigation of risk (deterministic bounding/failsafe operation) [23]. The MOD published JSP-936 [24] requires departmental and defence contractor adherence to governance around attributability, through life risk management and dependability assessments. All three of these publications support the safe utilisation of AI, recognising the need for additional verification of unique operational characteristics. The provision of such verification evidence requires modification to the traditional FEL, such that these activities can be costed and accounted for during scheduling.

The applicable means of compliance must be agreed and allocated within a defined safety case which safety critical software development already achieves through satisfaction of Def Stan 00-055 [4], and by extension DO-178C [1].

Human gated decision making is not indicative of any generalisable evidence of improved decision quality. Comparatively, the exclusion of required output from an LLMs response, is not evidence of the model's inability to provide such a response. Both scenarios share a common root cause, which is a lack of information and instruction. It is therefore imperative, that any UI design, accounts for a recognition that incorrect prompting can and will occur, information will be excluded, references to standards will be omitted and the full scope of the review may not be correctly specified. As part of agentic orchestration, or AI enablement, project configuration should be performed, such that if any of this information/context is excluded, either a safe approach to locating and using it should be identified, or the request should be blocked.

Parasuraman and Manzey [10] distinguish commission errors, where an incorrect automation recommendation is followed, from omission errors, where contradictory information is not sought. It is the principle of confirmation bias, which may prevent an originator from re-prompting the review of an artefact, if the model initially indicates no errors are present. Such eventualities must be avoided, using context injection, standardised artefact templating and process specific prompting.

Summary

Existing effort satisfactorily demonstrates the ability of LLM enablement for software engineering, and agentic tool utilisation in incremental development cycles, to a greater level of capability. Despite this, confirmation of AI/Agentic enabled DO-178C/DO-331 assured project execution, with the provision of independence and evidence, has not been demonstrated in the literature, at any significant DAL level [14-18].

Available benchmarks do not satisfactorily address the scope of the proposed investigation, therefore the identified gap, is a complete deployment and benchmarking of DO-178C compliant, MBSE integrated, LLM enabled processes, which are themselves extensible for use in an agentic orchestration [14-18]. The suggested approach separates the requirement for physical certification credit, from the engineering effort required to achieve the desired outcomes, therefore a reduced software implementation benchmark must be considered.

Methodology Reviews

Scope of Applicability

Two methodologies are compared against the traditional and approaches identified in the literature, to the exaction of the full engineering lifecycle (FEL) of a product. According to the scope of this report, RTCA DO-178C [1] remains applied as the basis for the adaptation of any process, particularly section 11, which defines the software lifecycle data.

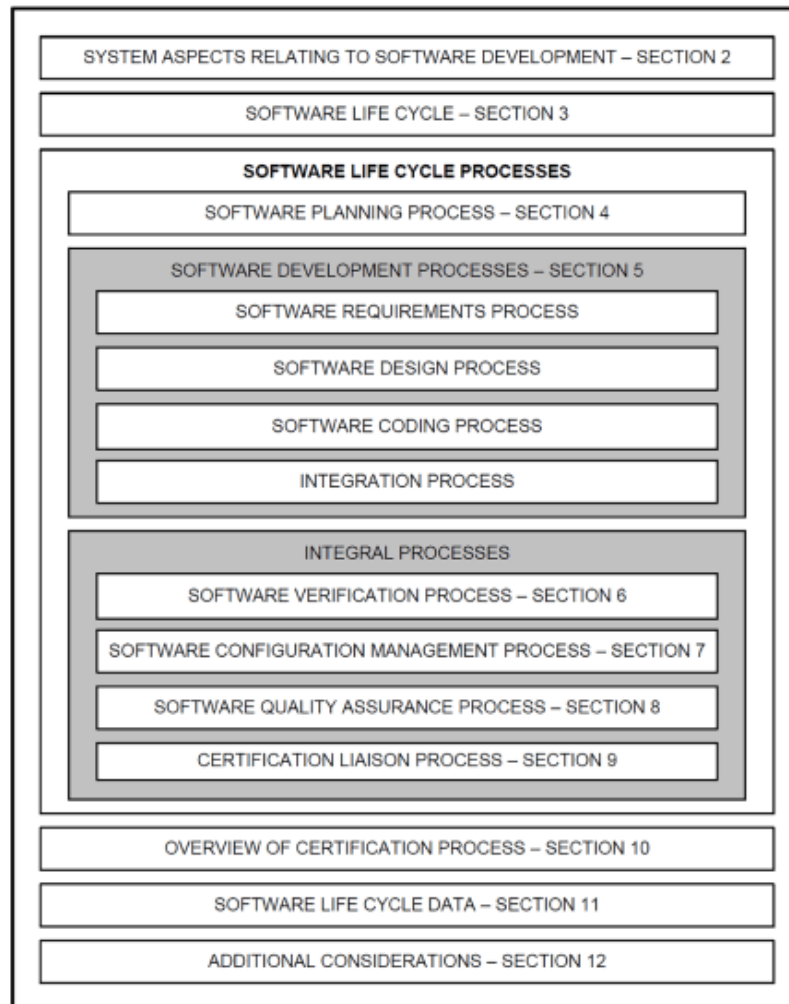


Figure 1 - RTCA DO-178C

To ensure adherence to DO-178C [1] remains viable through the transformation of any traditional process, Section 5 and 6 of Figure 1 are captured within the FEL illustrations below. All artefact associated processes required as per the allocated DAL rating, are preserved.

Both shall be assessed according to a non-inferiority investigation [28], where compliance to standards, quality, safety, traceability and independence must not fall below predefined acceptance margins. For both methodologies, all lifecycle data shall be treated as untrusted tool output, receiving no certification credit, and must be assumed into the requirement/standards baseline post independent SQEP verification and agreement.

In this assessment, it is a requirement that Change Control Board (CCB) measures are applied, to retain meaningful modification to requirement baselines, through submission of Software Problem Reports (SPRs) [1, 4].

Information categorisation:

Open

Methodology Control: Traditional FEL

A control is provided here, in the representation of the traditional FEL, which has been successfully utilised to produce certifiable software, across evolutions of the lifecycles structure, according to the standards selected, to form a software assurance case and to substantiate the means of certification.

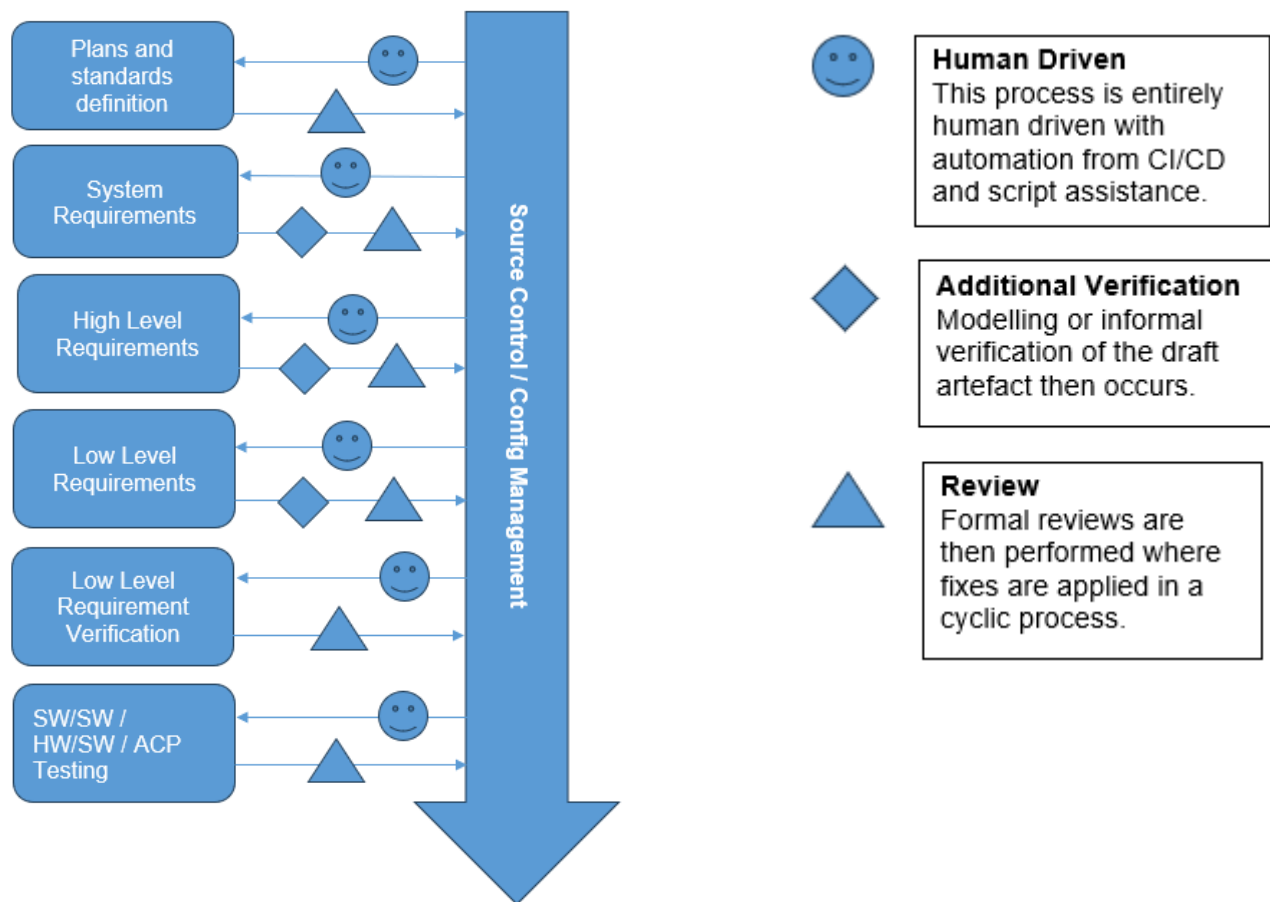


Figure 2 - Traditional FEL Illustration

Plans and standards as documented in 11.1 – 11.8 of Table 1 RTCA DO-178C Lifecycle Components are first created, reviewed and configured. From 11.9 – 11.16, requirements are defined and functional artefacts begin to be created, documented, traced, reviewed, verified and configured. This cycle repeats for each level of requirement decomposition, with baselining occurring at major review stages. Finally, from 11.17 – 11.22, defects are identified, resolved and baselined artefacts are collated, evaluated and configured [1, 3-5].

Failure Modes and Disadvantages

Human omission or misinterpretation can introduce incorrect requirements, model implementations, or verification implementations, propagating error through requirement baselines if not identified.

Manual traceability can become outdated with each commit as code is modified, the MBSE toolset mitigates this risk in part, but not where manual code is concerned [1, 6, 7]. Additionally, defects may remain unidentified until late in the lifecycle, or after requirement decomposition, with rework cycles introducing significant delays, costing more as software increases in complexity.

Information categorisation:

Open

Methodology Review 1: LLM Supported Traditional FEL

Established FELs already achieve the delivery of executable software and of all associated assurance material. Therefore, LLM enablement of these existing processes can be implemented without warranting the replacement of the lifecycle, or by effecting attributability, through adaption.

Process Architecture

A high-level definition of an LLM enabled software lifecycle is proposed below.

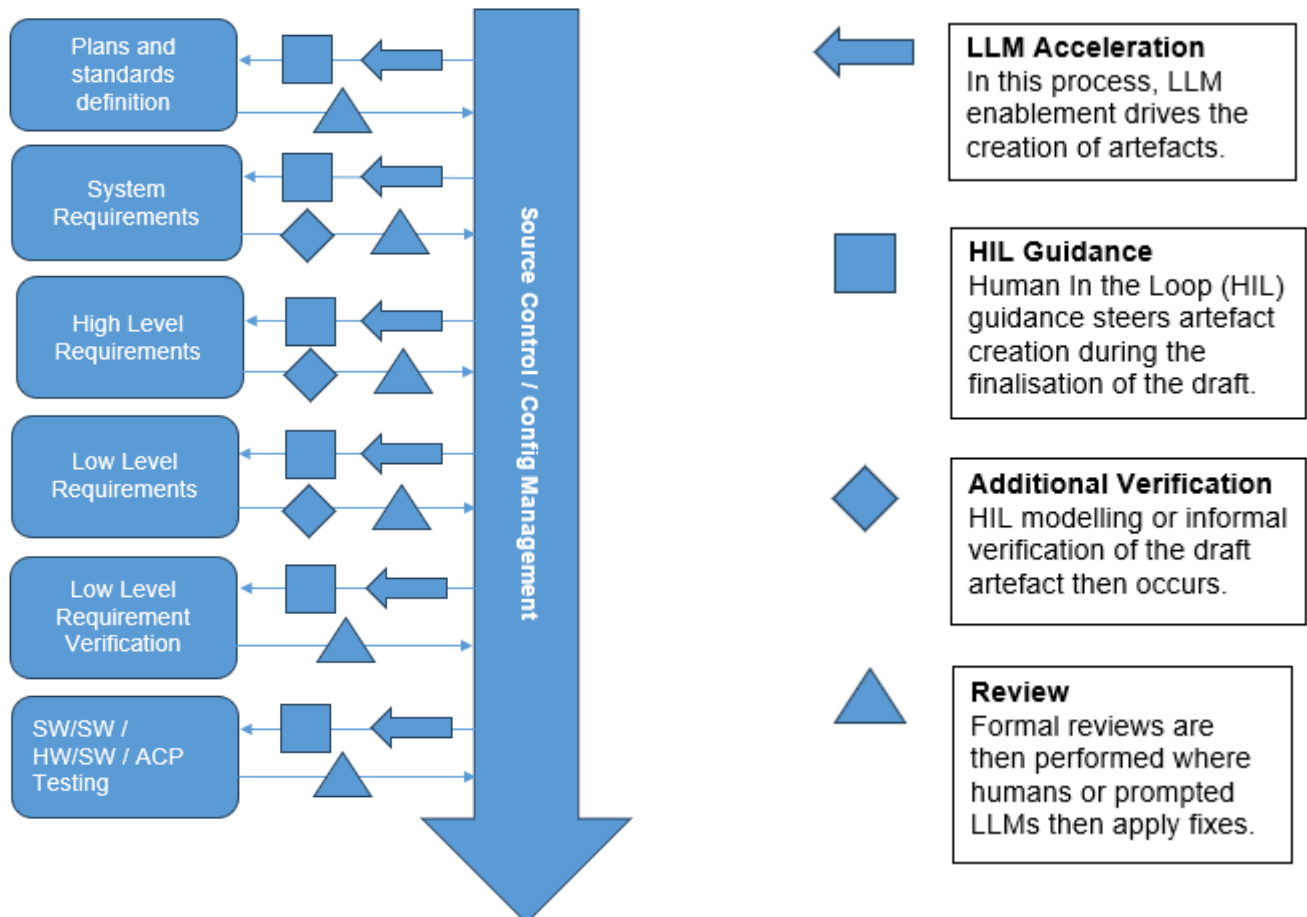


Figure 3 - LLM Enabled FEL Illustration

Assumptions and Prerequisites

This LLM Enabled FEL, adapted from existing examples of DAL A compatible approaches [1, 3] is driven by LLM generation, HIL guidance, verification, modelling and review. The production of traditional FEL associated artefacts requires SQEP, project context, tool knowledge and intimate knowledge of plans and standards. Through token sensitive orchestration, this information can be provided using Retrieval Augmented Generation (RAG), or prompting, thereby accelerating any data dependent process. RAG provides retrieved evidence, against which responses can be grounded, where responses must include accurate reference to any associated requirement, or stipulation within plans and standards documentation [8, 21].

Credibility of Approach

Figure 3 represents the minimum adjustment required to integrate this capability into existing processes. The allocation of human guidance is necessary to resolve ambiguity in requirements, interrupt incorrect generation and to prevent unnecessary work progressing until deployment.

Information categorisation:

Open

On premises LLM deployments, are constrained according to hardware limitations such as memory capacity and throughput, load balancing to permit parallel user access can further constrain model sizes and memory availability. Quantisation may reduce such requirement, by lowering the precision associated with weights and bias values [29]. Therefore, frontier model utilisation will be considered as a secondary control when estimating the efficacy of local deployment, accounting for the security boundary extension.

Failure Modes and Disadvantages

LLMs can misinterpret instruction, omit built in constraints and plausibly present incorrect output, introducing defects into requirement baselines if unchecked. In addition to human gated review, the correction of low certainty, through output repair looping, can validate output for correctness, potentially mitigating introduced defects [26]. Incomplete or corrupted data repositories may additionally introduce hallucinations, therefore only reviewed, issued and baselined material should enter the RAG corpus [8, 27].

The speed with which the production of artefacts is possible may increase the risk of defect propagation, but human gated acceptance, where review is the primary role of the engineer in these circumstances, can be employed to mitigate this early in the lifecycle [10]. The SQEP review requirement defined under the Scope of Applicability therefore applies for each configured artefact.

IP protection becomes difficult unless the model is severed from the internet, the entirety of the classified knowledge corpus must be stored locally to the hosting solution, to meet data handling requirements. This problem is compounded as external model hosting providers may incorrectly handle or expose project information, violating these rules. Sensitive projects must therefore agree an acceptable boundary of deployment, with agreed toolsets, secure by design principles, whilst abiding by model licensing restrictions [21, 22].

Lifecycle Modifications

This approach introduces effort for model hosting, selecting and defining the project information corpus, and artefact/process specific prompt creation, all upfront. Artefact drafting and rapid prototyping may see acceleration, but review effort is likely to increase, if output is unnecessarily complex, and if output is not properly evaluated, or tested.

The traditional FEL approaches of attributability can be easily maintained, therefore a standard matrix can be produced. An identifiable hazard here, is the reuse of LLMs as implementors, and reviewers. Current attribution methods and review roles can be retained; however, models cannot be differentiated as independent actors and are therefore, subject to the defined stipulations around human based independent verification [1, 2, 5].

Metrics and Acceptance Criteria

This methodology is selected as a stepwise evaluation, for the measurement of LLM acceleration. Tasking and implementation order shall be varied, so as not to invoke learned experience in the creation of artefacts. None of the LLM, the originator, or the reviewer shall have access to manual benchmark artefacts. Measurable characteristics allocated to each artefact are defined within the Experiment Design section. LLM specific measures will additionally include prompt cycles, token counts, RAG performance and the extent of manual correction performed.

Quality measures will include establishment of the removal of severity weighted defects, violations of plans and standards, requirement ambiguity, traceability precision and the extent of tool integration. The repeatability or determinism of results shall be established according to a repeat execution of a selection of tasks; it would be infeasible to perform multiple project implementations using LLM enablement alone.

The performance of the RAG capability shall be established separately, to ensure a bottleneck in context awareness is not introduced. The same RAG capability shall be integrated into the agentic orchestration capability, such that no associated tool quality bias is introduced.

Summary and Conditions of Rejection

The LLM supported FEL represents a direct approach to the preservation of attributability, whilst enabling measurement of the effect of LLM support. No presumption of safety or security benefit can be made without analysis of artefacts against stated requirements. According to the utilisation of human review, steering and accountability, existing review and verification measures are expected to mitigate any potential defect introduced by such enablement. Should this claim not be quantifiably substantiated, or should bottlenecks in this process not be identified, the Agentic FEL cannot be justified.

Methodology Review 2: Agentic Team Based FEL

Agentic automation presents greater theoretical concurrency, than LLM enablement, but requires greater modification to the FEL. The need for human in the loop validation and the ability to steer or reject activity remains necessary, and as such, gateways are inserted into the modified FEL.

Process Architecture

A high-level definition of this lifecycle is proposed below.

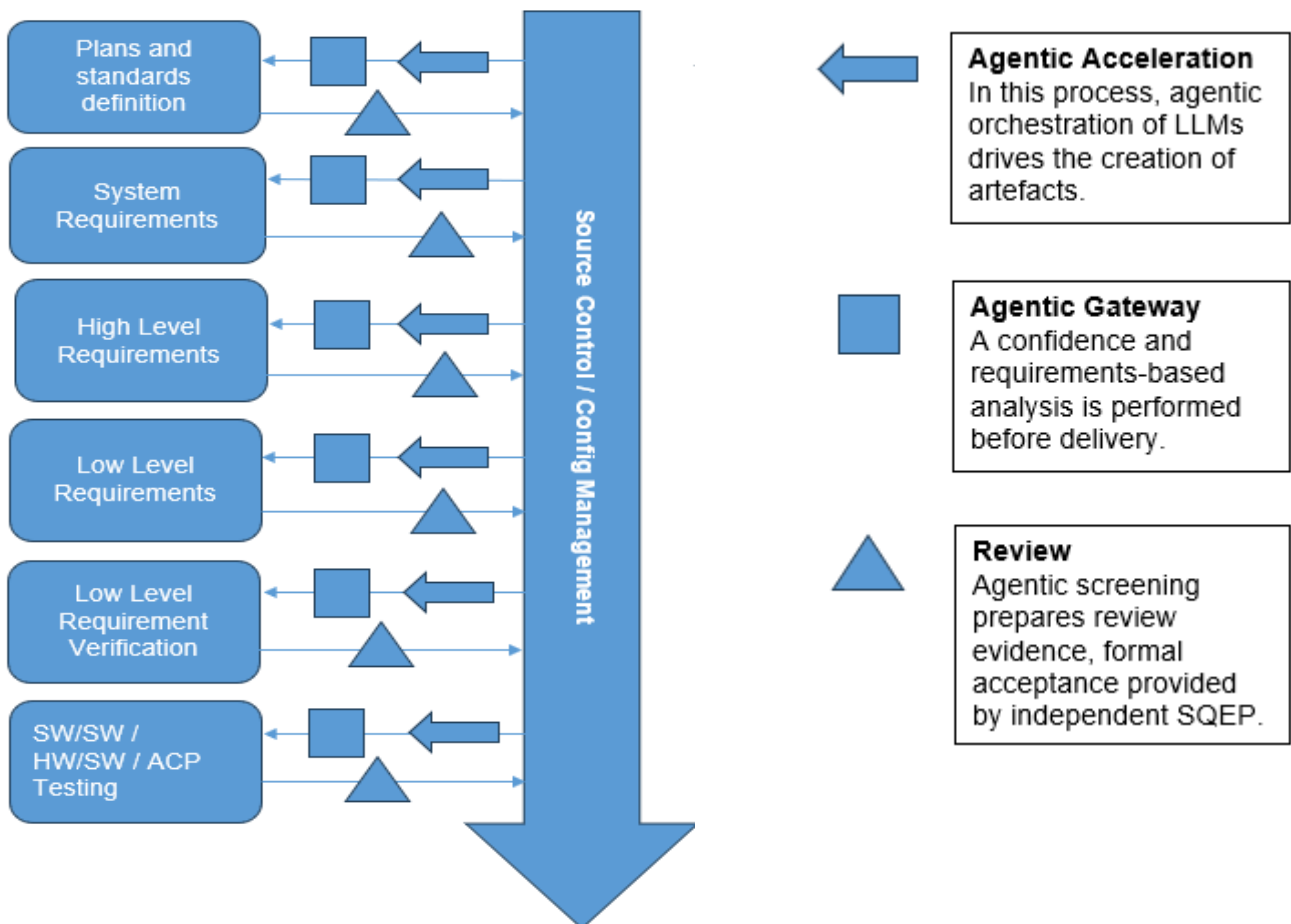


Figure 4 - Agentic FEL Illustration

Assumptions and Prerequisites

Here, immediately after a requirements baseline is issued, parallel completion of model implementation, informal tests, simulation verification, and traceability analysis can be commenced [16-18]. Gated confirmation stages, where human in the loop steering can be utilised if required, serve as incremental controllers for the acceptance of an artefact. Should the requirements of the task not be met according to an independent verifying agent, or human, incremental development can then be employed to address rework [25, 26].

Credibility of Approach

The safe parallelisation of tasks is a desired outcome of most FEL, however the independent verification requirement, disallows any agentic self-validating approach. The use of separate models does not establish verification independence; any acceptance must be performed by the authorised SQEP who is allocated the responsibility.

Information categorisation:

Open

The burden of assurance associated with this approach could be high; therefore, the quality of the plans and standards documentation must be absolute. Agentic output must still be treated as unqualified tool output, and any automated review must be substantiated by human approval, therefore human attribution is retained.

Failure Modes and Disadvantages

Incorrect assumptions introduced into SRATS could propagate through implementation and verification if not caught by a human, especially when reasoning generated during task completion is retained or reused. System level model simulation is performed to validate SRATS, derisking any following requirement decomposition. Model simulation at the HLR level becomes more valuable in this FEL, as decomposition validation can be achieved through use case associated interaction [17, 18]. Additionally, initiation of requirement decomposition can only be triggered after SQEP acceptance of the previous set of requirements, potentially reducing anticipated time savings.

Repeatability can be improved in an iterative manner, through model versioning, configurations, explicit prompting and bounded output, traceable back to requirements, however these measures don't prove model determinism [21]. Successful integration with software lifecycle toolsets is a key requirement, including a need for source control and local sandboxing, should any auditable rollback or recovery capability be achievable.

Lifecycle Modifications

Orchestration measures allow dependent work to be automatically handed off for human reviews, whilst progressing implementation for independent artefacts concurrently. This removes associated delays, however, introduces the risk of accelerated error, or the baselining of an incorrect assumption. The common human acceptance gates therefore remain necessary.

Successful agentic capability requires monitored and controlled interfacing with SCADE, Git, document creation tools, the operating system and web browsing, as shown by adjacent investigation [19, 21, 22, 25]. Web access should be disabled unless explicitly required and accounted for within any safe by design assessment. The experiment must consider utilisation of an existing orchestration framework, should such integration effort be commutable.

Automatic measures for the detection of unauthorised baseline modification, untraceable delivery, or non SPR/SCA linked change shall be implemented to prevent illegal change. Where such issues would be dealt with in team meetings, the associated lessons shall be incorporated into model instruction to model informal feedback. This would incur maintenance debt over time until a generalised, or specialised (as desired, per the basis of the task) set of instructions is reached.

Metrics and Acceptance Criteria

Model specialisation allows for the measurement of specific task performance, according to role, in addition to the common quality and timing metrics. The orchestration model, responsible for marshalling agent deployment and gauging task success, must permit interruption according to lifecycle review gates, where the review agent shall establish this quality. The verifying agent may therefore reject or return an artefact before the human gate is reached, however cannot provide acceptance or approval, lifecycle artefact acceptance remains the responsibility of independent SQEP.

Quality measures will additionally include the extent of agentic tool integration capability. The repeatability of results shall be established according to an automatic, repeat execution of a selection of tasks, with the potential for the project to be recreated and compared for the quantification of difference.

Summary and Conditions of Rejection

The execution of the Agentic FEL offers the greatest theoretical reduction in elapsed time, but also creates the largest assurance, security and human factors change. Adoption of such an approach would require demonstration of an incremental improvement capability, to justify selection of this capability over utilisation of an LLM supported FEL [10, 12-14].

Should concurrency not be demonstrated, or if the achieved benefit does not mitigate coordination overhead, duplicated activity or infrastructure cost, rejection must ensue. Similarly, resilience to and recovery from stalling, or failed toolset actions is a requirement for acceptance.

Effective demonstration of non-inferiority and defect identification must be achieved, in addition to provable superiority in comparison to artefacts created using the control, or using an LLM supported FEL [28]. If such conditions cannot be met, and if improvements are not statistically meaningful, the safer FEL shall be selected.

Methodology Application and Future Development

Initial Benchmark

The next step in this analysis, involves the production of a controlled benchmark system using manual programming techniques. The chosen benchmark is a Command and Control system for an automatic drone, developed in SCADE Suite and Display, with 4 representative data panels. An MQTT based communication system shall emulate communication/control of a simulated remote platform. This system and its associated artefacts are designed to exercise a representative subset of DAL B objectives [1, 3].

The chosen example is realistically achievable in a shorter timeframe, because external dependencies are deliberately bounded or emulated, requirements are defined before implementation, and selected documentation may be stubbed. Schedule assumptions are reflective of experience in comparable SCADE based projects, including the expected review, process definition, implementation and integration efforts.

Traditional estimating techniques and experience were employed to design a schedule for the enaction of the traditional FEL, with literature-based schedule hypotheses formed for the additional FELs, included in Table 2 below. Peng et al. [11] identified a 55.8% reduced completion time, but for a non-comparative project scope, whereas Cui et al. [20] identified a 26.08% increase across comparable workspaces. Despite these improvements, acceleration of output does not itself indicate faster acceptance or review, with the opposing shift towards additional rework observed by Perry et al [13].

Should planned benefit be realised without considerable integration/accuracy/performance challenges, the 'Steady Progression' timings apply. 'Validated Use' considers time allocated for rework cycles, given literature reflected partial acceleration of artefact implementation/reviewing [11, 12].

Approach	Steady Progression	Validated Use (Incl rework)
Traditional FEL	42 weeks	42 weeks
LLM Enabled FEL	35 weeks	41 weeks
Agentic FEL	30 weeks	40 weeks

Table 2 - C2 Hypothesised Project Schedules

Experiment Design

An independently reviewable package of changes, produced and submitted for review forms the unit of analysis. Packages, classified by their comparative functional complexity shall be allocated for completion across the three lifecycles. The allocation and order of task implementation shall be randomised (per requirement decomposition stage) to reduce any learning effect with reviewers remaining uninformed as to the origination of the artefact.

The applied system of time logging should distinguish active engineering, context search, prompt construction, deterministic tool execution, model latency, review, rework and waiting time. Quality shall be assessed using functional correctness, robustness, model and coding standard compliance, verification adequacy, traceability precision and recall, completeness of lifecycle data, and unintended functionality.

The experiment shall utilise a non-inferiority approach, with margins pre-defined for each measurable quality metric before the examination of results, but as a minimum no seeded catastrophic or major defects must remain in the baseline [28].

Information categorisation:

Open

Table 3 illustrates the planned analyses to be undertaken against each methodology, and the control actions performed to produce a baseline for comparison. Here the human factors requirement was influenced by Parasuraman and Manzey's review [10]. Security assessment requirements shall be defined, according to the associated NIST publications [21, 22].

Analysis	Control Action	Methodology Assessment Action
Purpose Suitability	Establish baseline effort and identify common defects.	Measure effect of AI enabled (LLM) and incremental (agentic) approaches against baseline effort. Agentic approaches shall be tested according to deployed agentic team structure and size.
Timing Benchmark	Complete the engineering lifecycle using standard / stubbed methods, record timings.	Utilise each approach to expedite the lifecycle, record timings. Focus on acceleration for LLM enabled FEL and incremental automation for the agentic FEL.
Achievement	Identify baseline performance in traceability, functionality error, coverage, independent review.	Identify LLM enabled / agentic performance in traceability error, functionality error, coverage, independent review.
Failure Analysis	Perform a failure analysis resulting from seeded, intentional defects.	Identify successful/failed removal of seeded defects, record point of identification and removal.
Human Factors	Perform a baseline human factors assessment to establish good design principles of display components.	Perform the same assessment against LLM enabled and agentic designs. Ensuring the LLMs have knowledge of the assessment standards beforehand.
Security	Identify security requirements and ensure these are followed.	Record and configure evidence of adherence to security requirements, once created.
Decision	Identify which approach resulted in the fewest introduced defects, the highest requirement / implementation / verification quality, in the shortest timeframe.	

Table 3 - Methodology Analysis Matrix

Future Development and Assessment

DO-178C/DO-331 remains the primary acceptance basis [1, 3, 5] for plans and standards documentation and lifecycle data, with any other artefact reviewed against the plans and standards themselves. This investigation targets a DAL B level of assurance, therefore limited generalisable findings can be assumed for any other DAL. The benchmark cannot demonstrate compliance or assurance without engagement with authorities; therefore, its purpose remains limited to the quantification and analysis of lifecycle and process performance.

It is recognised this investigation is limited by its use of a synthetic and partially stubbed benchmark, additionally the experience of the originator, learning/ordering effects and the quality of the project information corpus may affect results.

According to the potential benefit, both methodologies will be evaluated through the performance of the stated experiment. The production of an empirical method for the establishment of holistic performance, once complete, will provide the necessary evidence to substantiate process transformation in this area. Engagement with the certification bodies, the identification of standards to cover usage of AI tooling, and proof of general applicability remain necessary for future adoption.

Information categorisation:

Open

Subject to the findings of the stated experiment, the least impactful change, LLM enablement, remains the most defensible position, for short term modification of the FEL via utilisation of existing governance structures, agreed processes and evidence collection methods.

Agentic orchestration constitutes a risky and experimental approach to DAL B safety critical process transformation, ill evidenced by the literature [14-18], whose benefit relies upon dependency ridden integration challenges, extensive orchestration [27] and proving security performance [21, 22].

References

Locally held copies of references [1 – 3] were utilised to produce this document, access to these standards requires purchase. The information cited is alternatively available online.

1. RTCA (2011a) *DO-178C: Software Considerations in Airborne Systems and Equipment Certification*. Washington, DC: RTCA, Inc. Available at: <https://www.rtca.org/do-178/>.
2. RTCA (2011b) *DO-330: Software Tool Qualification Considerations*. Washington, DC: RTCA, Inc. Available at: <https://my.rtca.org/productdetails?id=a1B36000001lcfkEAC>.
3. RTCA (2011c) *DO-331: Model-Based Development and Verification Supplement to DO-178C and DO-278A*. Washington, DC: RTCA, Inc. Available at: <https://my.rtca.org/productdetails?id=a1B36000001lcfiEAC>.
4. Ministry of Defence (2021) *Defence Standard 00-055 Part 1, Issue 5: Requirements for Safety of Programmable Elements (PE) in Defence Systems—Part 1: Requirements and Guidance*. Glasgow: UK Defence Standardization.
5. Federal Aviation Administration (2017) *AC 20-115D: Airborne Software Development Assurance Using EUROCAE ED-12() and RTCA DO-178()*. Washington, DC: FAA. Available at: https://www.faa.gov/regulations_policies/advisory_circulars/index.cfm/go/document.information/documentID/1032046.
6. Ansys (2025a) 'Efficient Development of Safe Avionics Software with DO-178C Objectives Using SCADE Suite', 9 April. Available at: <https://innovation.ansys.com/public/articles/topic/efficient-development-of-safe-avionics-software-with-do-178c-objectives-using-scade-suite-expu>.
7. Ansys (2025b) 'Efficient Verification of Safe Avionics Software with DO-178C Objectives Using SCADE Suite', 19 May. Available at: <https://innovation.ansys.com/public/articles/topic/efficient-verification-of-safe-avionics-software-with-do-178c-objectives-using-scade-suite-expu>.
8. Lewis, P. et al. (2020) 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks', *Advances in Neural Information Processing Systems*, 33, pp. 9459–9474. Available at: <https://arxiv.org/abs/2005.11401>.
9. European Union Aviation Safety Agency (2026) *EASA Artificial Intelligence Concept Paper—Proposed Issue 3*. Cologne: EASA. Available at: <https://www.easa.europa.eu/en/document-library/general-publications/easa-artificial-intelligence-concept-paper-proposed-issue-3>.
10. Parasuraman, R. and Manzey, D.H. (2010) 'Complacency and Bias in Human Use of Automation: An Attentional Integration', *Human Factors*, 52(3), pp. 381–410. <https://doi.org/10.1177/0018720810376055>.
11. Peng, S., Kalliamvakou, E., Cihon, P. and Demirer, M. (2023) 'The Impact of AI on Developer Productivity: Evidence from GitHub Copilot', *arXiv:2302.06590*. Available at: <https://arxiv.org/abs/2302.06590>.

12. Becker, J., Rush, N., Barnes, E. and Rein, D. (2025) 'Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity', arXiv:2507.09089. Available at: <https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/>.
13. Perry, N., Srivastava, M., Kumar, D. and Boneh, D. (2023) 'Do Users Write More Insecure Code with AI Assistants?', Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. <https://doi.org/10.1145/3576915.3623157>.
14. Graydon, M.S. and Lehman, S.M. (2025) Examining Proposed Uses of LLMs to Produce or Assess Assurance Arguments. NASA/TM-20250001849. Hampton, VA: NASA. Available at: <https://ntrs.nasa.gov/citations/20250001849>.
15. Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O. and Narasimhan, K. (2024) 'SWE-bench: Can Language Models Resolve Real-World GitHub Issues?', Proceedings of the Twelfth International Conference on Learning Representations. Available at: <https://arxiv.org/abs/2310.06770>.
16. Xia, C.S., Deng, Y., Dunn, S. and Zhang, L. (2025) 'Agentless: Demystifying LLM-based Software Engineering Agents', *Proceedings of the 33rd ACM SIGSOFT International Symposium on the Foundations of Software Engineering*. Available at: <https://arxiv.org/abs/2407.01489>.
17. Hong, S. et al. (2024) 'MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework', *Proceedings of the Twelfth International Conference on Learning Representations*. Available at: <https://arxiv.org/abs/2308.00352>.
18. Qian, C. et al. (2024) 'ChatDev: Communicative Agents for Software Development', *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 1, pp. 15174–15186. Available at: <https://arxiv.org/abs/2307.07924>.
19. Ansys (2024) 'Using the SCADE Python APIs from Your Favorite IDE', 3 January. Available at: <https://innovation.ansys.com/public/articles/topic/using-the-scade-python-apis-from-your-favorite-ide-expu>.
20. Cui, K.Z., Demirer, M., Jaffe, S., Musolff, L., Peng, S. and Salz, T. (2026) 'The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers', *Management Science*, advance online publication. doi:10.1287/mnsc.2025.00535.
21. National Institute of Standards and Technology (2024a) *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST AI 600-1. Gaithersburg, MD: NIST. Available at: <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>.
22. National Institute of Standards and Technology (2024b) *Secure Software Development Practices for Generative AI and Dual-Use Foundation Models: An SSDF Community Profile*. NIST SP 800-218A. Gaithersburg, MD: NIST. <https://doi.org/10.6028/NIST.SP.800-218A>.

23. Military Aviation Authority (2025) *MAA/RN/2025/04: Use of Artificial Intelligence within Systems and Equipment Subject to MRP Design Assurance Processes*. Bristol: Ministry of Defence. Available at: https://assets.publishing.service.gov.uk/media/68f0b97c1c9076042263ef2a/MAA_RN_2025_04.pdf
24. Ministry of Defence (2024) *JSP 936: Dependable Artificial Intelligence in Defence—Part 1: Directive*. London: Ministry of Defence. Available at: <https://www.gov.uk/government/publications/jsp-936-dependable-artificial-intelligence-ai-in-defence-part-1-directive>.
25. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K.R. and Cao, Y. (2023) 'ReAct: Synergizing Reasoning and Acting in Language Models', *Proceedings of the Eleventh International Conference on Learning Representations*. Available at: <https://arxiv.org/abs/2210.03629>.
26. Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K.R. and Yao, S. (2023) 'Reflexion: Language Agents with Verbal Reinforcement Learning', *Advances in Neural Information Processing Systems*, 36. Available at: <https://arxiv.org/abs/2303.11366>.
27. Zou, W., Geng, R., Wang, B. and Jia, J. (2025) 'PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models', *Proceedings of the 34th USENIX Security Symposium*, pp. 3827–3844. Available at: <https://www.usenix.org/conference/usenixsecurity25/presentation/zou-poisonedrag>.
28. Lakens, D. (2017) 'Equivalence Tests: A Practical Primer for t Tests, Correlations, and Meta-Analyses', *Social Psychological and Personality Science*, 8(4), pp. 355–362. <https://doi.org/10.1177/1948550617697177>.
29. Dettmers, T., Lewis, M., Belkada, Y. and Zettlemoyer, L. (2022) 'LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale', *Advances in Neural Information Processing Systems*, 35. Available at: <https://arxiv.org/abs/2208.07339>.